Write-up: Ransomware

por *dleskow*

Informações do chall

- **Título**: Reversin.Kr/Ransomware
- **Autor**: Pyutic
- **Contexto**: Decrypt File (EXE)
- **Arquivos**:
  - *ransomware.zip*
    - *run.exe*
    - *file*
    - *readme.txt*

1. Write-Up

1.1. Análise Estática

Primeiro rodamos um `file` no arquivo:

```
┌──(dleskow㉿ganesh)-[/home/ganesh/Reversin.Kr/Ransomware]
└─# file run.exe
run.exe: PE32 executable for MS Windows 5.01 (console), Intel i386, UPX compressed, 3 sections
```

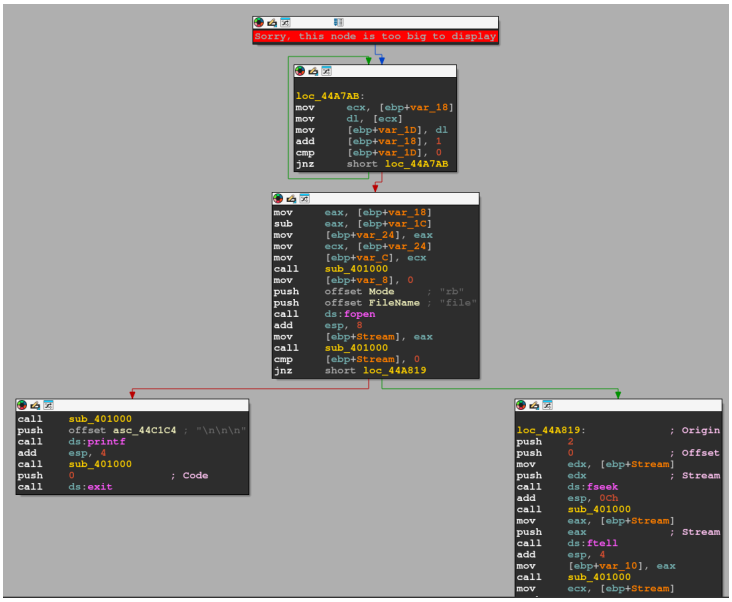
Como ele foi comprimido usando UPX, a decompressão é trivial:

```
┌──(dleskow㉿ganesh)-[/home/ganesh/Reversin.Kr/Ransomware]
└─# upx -d run.exe
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2024
UPX 4.2.4       Markus Oberhumer, Laszlo Molnar & John Reiser    May 9th 2024

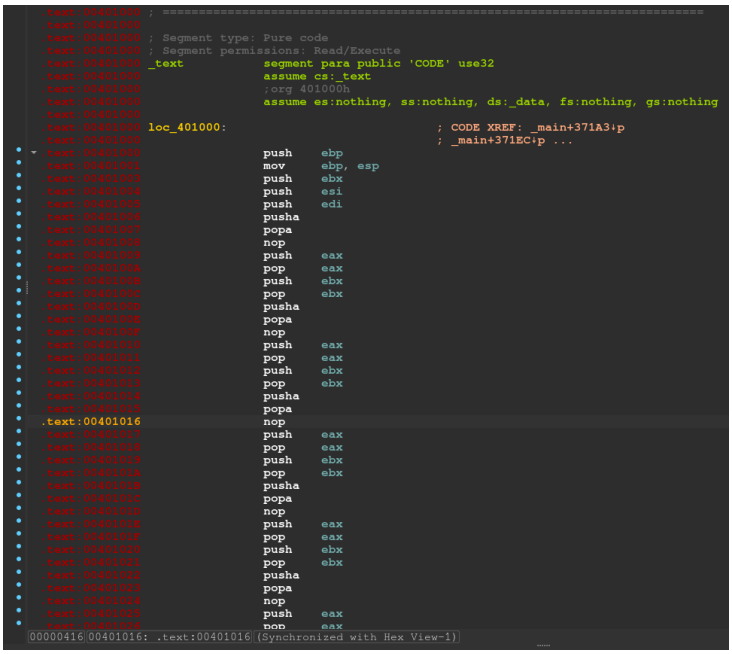
        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
    311296 <-     10240    3.29%    win32/pe     run.exe

Unpacked 1 file.
```

Abrindo no IDA o arquivo resultante, nos deparamos com o IDA dizendo que um dos nós é muito grande para ser exibido,e qualquer tentativa de decompilação repetirá o mesmo erro:



Se tentarmos ver o que há nessa função, seja pelo `objdump` ou pelo IDA, vemos que é apenas muito (muito) lixo, absolutamente inútil:



Mas o que nos garante que esta função é realmente só lixo? Já mostro isso, apenas ir para o GDB antes.

No caso, isto é agora, pois, por incompetência minha com o IDA ao lidar com quantidades exorbitantes de lixo, fiz todo o resto no GDB.

1.2. GDB

Programa normal

Como Windows não é algo amigável, e ainda não me foi necessário fazer nada relevante lá, usarei o wine para executar o programa. Nele, também anexarei o GDB (pwndbg, na verdade) para podermos analisar o programa.

Antes, porém, vejamos uma execução usual dele:

```
┌──(dleskow㉿ganesh)-[/home/ganesh/Reversin.Kr/Ransomware]
└─# wine run.exe
Key : abcde
```

Ou seja, ele printa um monte de lixo, pede uma chave, printa mais lixo e encerra o programa. Até agora não consegui descobrir se esse lixo era para ser algo (não descobri nada em coreano).

Agora, rodemos o programa no GDB:

Função cheia de lixo

```
┌──(dleskow㉿ganesh)-[/home/ganesh/Reversin.Kr/Ransomware]
└─# winedbg --gdb run.exe
```

Aproveitando que já chegamos no GDB, aproveitemos e mostremos como analisar aquele lixo de forma simples. Queremos jogar todas aquelas linhas de lixo, lidas como assembly, para um arquivo, onde poderemos apagar o lixo “na mão”.

Para tal, ativamos o `logging` no GDB e mandamos dar disassemble na região que contém o lixo (os endereços mais exatos podem ser obtidos no IDA):

```
pwndbg> set logging file weird_fn.asm
pwndbg> set logging enabled on
Copying output to weird_fn.asm.
Copying debug output to weird_fn.asm.
pwndbg> disassemble 0x401000, 0x44a775
...

pwndbg> set logging enabled off
Done logging to weird_fn.asm.
```

Agora que temos toda a função em um arquivo, utilizamos a maravilha do RegEx, combinado ao `neovim` (qualquer lugar com busca e substituição por RegEx serve). Perceba que todas as linhas que são certamente inúteis são as que dão `push` de algo e, na linha imediatamente após, dão `pop` do mesmo. No caso, temos 3 tipos de linhas que podemos excluir sem dó:

- 1: pusha, popa
- 2: push $reg, pop $reg
- 3: nop

Onde $reg é qualquer registrador. Não explicarei muito as ReGex utilizadas, mas, essencialmente, elas procuram por duas linhas consecutivas que deem push e pop da mesma coisa. O resto é sintaxe do neovim para “busque e substitua” ou “substitua isso por nada”, aka excluir isso tudo:

```
1: :%s/^.*push\(.*\)$\n.*pop\1\n/
2: :%s/^.*push\s*\(\w*\)$\n.*pop\s*\1\n/
3: :%s/^.*nop.*$\n/
```

O que resta, surpreendentemente, não é um arquivo vazio:

```
   0x00401000:  push   ebp
   0x00401001:  mov    ebp,esp
   0x00401003:  push   ebx
   0x00401004:  push   esi
   0x00401005:  push   edi
   0x004135ce:  pop    edi
   0x004135cf:  pop    esi
   0x004135d0:  pop    ebx
   0x004135d1:  pop    ebp
   0x004135d2:  ret

   0x004135d3:  int3
   0x004135d4:  int3
   0x004135d5:  int3
   0x004135d6:  int3
   0x004135d7:  int3
   0x004135d8:  int3
   0x004135d9:  int3
   0x004135da:  int3
   0x004135db:  int3
   0x004135dc:  int3
   0x004135dd:  int3
   0x004135de:  int3
   0x004135df:  int3
   0x004135e0:  push   ebp
   0x004135e1:  mov    ebp,esp
   0x004135e3:  sub    esp,0x24
   0x004135e6:  push   ebx
   0x004135e7:  push   esi
   0x004135e8:  push   edi

   0x00425bb1:  pusha
   0x00425bb2:  mov    eax,fs:0x18
   0x00425bb8:  mov    eax,DWORD PTR ds:[eax+0x30]
   0x00425bbc:  mov    BYTE PTR ds:[eax+0x2],0x44
   0x00425bc1:  popa
   0x00425bc2:  push   0x0
   0x00425bc4:  push   0x0
   0x00425bc6:  push   0x0
   0x00425bc8:  push   0x44a990
   0x00425bcd:  push   0x0
   0x00425bcf:  push   0x0
   0x00425bd1:  call   DWORD PTR ds:0x44c000

   0x0043819f:  push   0x44c118
```

```
   0x004381a4:  call   DWORD PTR ds:0x44c0b0
   0x004381aa:  add    esp,0x4
```

Temos aí três coisas:

- `ret` –> Logo, qualquer chamada a 0x401000 vai retornar sem fazer nada além dos `push` e `pop` inúteis;
- Uma `call` que chama uma Thread: não é relevante, pelo que vi;
- Uma `call` ao print: é o que printa aquele lixo inicial (localizado em 0x44c118).

Análise Dinâmica

Bem, agora façamos, de fato, a análise dinâmica no GDB. Como sabemos que ele começa printando o lixo antes de pedir uma chave, podemos começar a análise no `printf`. Além disso, sugiro aumentar o número de linhas no contexto, para melhor visualização.

```
pwndbg> b* printf
Breakpoint 1 at 0x798056b4
pwndbg> set context-disasm-lines 50
Set number of additional lines to print in the disasm context to 50.
pwndbg> c
```

Aí vemos o backtrace e analisamos para a função anterior, que chamou o printf:

```
──────────────────────────[ BACKTRACE ]──────────────────────────
 ► 0 0x798056b4 printf
   1 0x4381aa None
   2 0x44ab7a None
   3 0x7bc09ab0 WriteTapemark@16+256
   4 0x7bd98203 call_thread_func_wrapper+23
   5 0x7bd98c75 call_thread_func+117
   6      0x0 None

pwndbg> x/10i 0x43819d
   0x43819d:    push   ebx
   0x43819e:    pop    ebx
   0x43819f:    push   0x44c118  // String esquisita
   0x4381a4:    call   DWORD PTR ds:0x44c0b0  // printf
   0x4381aa:    add    esp,0x4  // Limpa arg
   0x4381ad:    pusha
   0x4381ae:    popa
   0x4381af:    nop
   0x4381b0:    push   eax
   0x4381b1:    pop    eax

pwndbg> x/2s 0x44c118
0x44c118:       "\263\252\264\302 \263\252\273r\360\300\331!\n\263\252\264\302 \270f\354 \263\252\273241\277\256\271\247\266\ 342\گ
317\276\ 273\300\317\300\304\306\ 307\300\312\263\x8õ\307331\'!\n\263\312\300\307 \306\304\300\317\300\273 \272\271\261\270\307x\355 \275'351\ئ
5ô\276\357 \264\336\267\257\270\246 \300U\335\307x\355 \271\336\300\272 Ü\260\252\300\270\267\316 \306\304\300\317\300\273
\272\271\261\270\307363\ص!\n\n"
0x44c1ac:       "Key : "
```

Aqui vemos a string esquisita lá, e vemos que temos a seção das strings do código, uma vez que "Key : " está logo subsequente.

Agora, vejamos qual o próximo passo do código. `nextcall` é um comando maravilhoso, mas pode levar um tempinho (30s) até ele achar a próxima `call` e parar:

```
pwndbg> nextcall
-------------------------[ DISASM / i386 / set emulate on ]-------------------------
   0x44a773    push   ebx
   0x44a774    pop    ebx
   0x44a775    push   0x44c1ac        // "Key : "
 ► 0x44a77a    call   dword ptr [0x44c0b0]        // FUNÇÃO: printf

   0x44a780    add    esp, 4
   0x44a783    call   0x401000        <0x401000>
```

Aqui ele printa "Key : ". Como a função `0x401000` já foi averiguada inútil, simplesmente a ignorarei daqui para a frente. Para poder pulá-la durante o GDB, apenas porei um `breakpoint` no `ret` dela. Mostrarei apenas esta primeira vez, as próximas ficam subentendidas que apenas ignorei esta função.

Daqui para a frente, realcei as linhas mais relevantes (que mexem com input ou chamam funções) usando ► e coloquei comentários (em pseudo-C, ±) para ajudar a entender o que está acontecendo.

```
pwndbg> b* 0x004135d2
Breakpoint 2 at 0x4135d2
pwndbg> c
-------------------------[ DISASM / i386 / set emulate on ]-------------------------
► 0x44a788   push   0x44d370        // input (em ascii), chamemos seu tamanho de l
  0x44a78d   push   0x44c1b4
  0x44a792   call   dword ptr [0x44c0b8]        // FUNÇÃO: scanf

  0x44a798   add    esp, 8
► 0x44a79b   mov    dword ptr [ebp - 0x18], 0x44d370        // Basicamente, contador i = 0
  0x44a7a2   mov    eax, dword ptr [ebp - 0x18]        // eax <- &input[0]
  0x44a7a5   add    eax, 1        EAX => 0x44d371 (0x44d370 + 0x1)  // eax <- &input[1]
  0x44a7a8   mov    dword ptr [ebp - 0x1c], eax        [0xfcfee0] <= 0x44d371  // ebp-0x1c <- &input[1]
  0x44a7ab   mov    ecx, dword ptr [ebp - 0x18]        ECX, [0xfcfee4]         // ecx <- &input[0]
► 0x44a7ae   mov    dl, byte ptr [ecx]        DL, [0x44d370]          // dl <- input[0]
► 0x44a7b0   mov    byte ptr [ebp - 0x1d], dl        [0xfcfedf] <= 0x61      // ebp-0x1d <- input[0]
  0x44a7b3   add    dword ptr [ebp - 0x18], 1        // i += 1
► 0x44a7b7   cmp    byte ptr [ebp - 0x1d], 0        EFLAGS => 0x202 [ cf pf af zf sf IF df of ac ]  // input[0] == \0
  0x44a7bb ✔ jne    0x44a7ab        <0x44a7ab>  // loop 1 vez
    ↓
  0x44a7ab   mov    ecx, dword ptr [ebp - 0x18]        ECX, [0xfcfee4]     // ecx <- &input[i]
  0x44a7ae   mov    dl, byte ptr [ecx]        DL, [0x44d371]      // dl <- input[i]
  0x44a7b0   mov    byte ptr [ebp - 0x1d], dl        [0xfcfedf] <= 0x62  // ebp-0x1d <- input[i]
  0x44a7b3   add    dword ptr [ebp - 0x18], 1        // pos += 1
► 0x44a7b7   cmp    byte ptr [ebp - 0x1d], 0        EFLAGS => 0x202 [ cf pf af zf sf IF df of ac ]  // while (input[i] != 0)
  0x44a7bb ✔ jne    0x44a7ab        <0x44a7ab>  // loop de novo
    ↓
... (loop) // --> Percorre nosso input e vai até o fim dele
    ↓
► 0x44a7b7   cmp    byte ptr [ebp - 0x1d], 0        EFLAGS => 0x246 [ cf PF af ZF sf IF df of ac ]  // Fim da string
  0x44a7bb ✘ jne    0x44a7ab        <0x44a7ab>

  0x44a7bd   mov    eax, dword ptr [ebp - 0x18]        EAX, [0xfcfee4]  // eax <- &input[l]
  0x44a7c0   sub    eax, dword ptr [ebp - 0x1c]        EAX => 5         // eax <- &input[l] - &input[0] = l
  0x44a7c3   mov    dword ptr [ebp - 0x24], eax        [0xfcfed8] <= 5  // ebp-0x24 = l
  0x44a7c6   mov    ecx, dword ptr [ebp - 0x24]        ECX, [0xfcfed8]  // ecx <- l
  0x44a7c9   mov    dword ptr [ebp - 0xc], ecx        [0xfcfef0] <= 5  // ebp-0xc <- l
  0x44a7cc   call   0x401000        <0x401000>
```

Ou seja, esta parte do código foi, basicamente, um `strlen(input)`, e ele salva o tamanho `l` de nosso input em `ebp-0xc`. Continuemos o programa.

```
-------------------------[ DISASM / i386 / set emulate on ]-------------------------
  0x44a7d1   mov    dword ptr [ebp - 8], 0        [0xfcfef4] <= 0
  0x44a7d8   push   0x44c1b8        // "rb"
  0x44a7dd   push   0x44c1bc        // "file"
► 0x44a7e2   call   dword ptr [0x44c0ac]        // FUNÇÃO: fopen

  0x44a7e8   add    esp, 8
  0x44a7eb   mov    dword ptr [ebp - 4], eax        // ebp-0x4 <- FILE*
  0x44a7ee   call   0x401000        <0x401000>


  0x44a7f3   cmp    dword ptr [ebp - 4], 0        EFLAGS => 0x202 [ cf pf af zf sf IF df of ac ]  // FILE* != NULL
  0x44a7f7 ✔ jne    0x44a819        <0x44a819>
    ↓
  0x44a819   push   2        // SEEK_END
  0x44a81b   push   0        // distância de SEEK_END
  0x44a81d   mov    edx, dword ptr [ebp - 4]        EDX, [0xfcfef8]
  0x44a820   push   edx        // FILE*
► 0x44a821   call   dword ptr [0x44c0a4]        // FUNÇÃO: fseek

  0x44a827   add    esp, 0xc        // Limpa 3 args da stack
  0x44a82a   call   0x401000        <0x401000>


  0x44a82f   mov    eax, dword ptr [ebp - 4]        EAX, [0xfcfef8]
  0x44a832   push   eax        // FILE*
► 0x44a833   call   dword ptr [0x44c0a8]        // FUNÇÃO: ftell

  0x44a839   add    esp, 4        // Limpa arg
  0x44a83c   mov    dword ptr [ebp - 0x10], eax
  0x44a83f   call   0x401000        <0x401000>


  0x44a839   add    esp, 4        ESP => 0xfcfecc (0xfcfec8 + 0x4)
  0x44a83c   mov    dword ptr [ebp - 0x10], eax        [0xfcfeec] <= 0x2400  // ebp-0x10 <- 0x2400 = tamanho do "file"
  0x44a83f   call   0x401000        <0x401000>

  0x44a844   mov    ecx, dword ptr [ebp - 4]        ECX, [0xfcfef8]
  0x44a847   push   ecx        // FILE *
► 0x44a848   call   dword ptr [0x44c0bc]        // FUNÇÃO: rewind

  0x44a84e   add    esp, 4        ESP => 0xfcfecc (0xfcfec8 + 0x4)  // Limpa args
  0x44a851   call   0x401000        <0x401000>
```

Essa parte abre o arquivo, guarda seu ponteiro em `ebp-0x4`, pega o tamanho do arquivo (0x2400) e guarda em `ebp-0x10` e volta para o início do arquivo. Continuemos.

```
-------------------------[ DISASM / i386 / set emulate on ]-------------------------
  0x44a856   mov    edx, dword ptr [ebp - 4]        EDX, [0xfcfef8]
  0x44a859   push   edx        // FILE*
► 0x44a85a   call   dword ptr [0x44c0c0]        // FUNÇÃO: feof

  0x44a860   add    esp, 4        ESP => 0xfcfecc (0xfcfec8 + 0x4)  // Limpa arg
  0x44a863   test   eax, eax        0 & 0     EFLAGS => 0x246 [ cf PF af ZF sf IF df of ac ]
► 0x44a865 ✘ jne    0x44a897        <0x44a897>  // Não pula, pois ainda não é EOF

  0x44a867   call   0x401000        <0x401000>

  0x44a86c   mov    eax, dword ptr [ebp - 4]        EAX, [0xfcfef8]
  0x44a86f   push   eax        // FILE*
► 0x44a870   call   dword ptr [0x44c0b4]        // FUNÇÃO: getc

  0x44a876   add    esp, 4        ESP => 0xfcfecc (0xfcfec8 + 0x4)  // Limpa arg
  0x44a879   mov    ecx, dword ptr [ebp - 8]        ECX, [0xfcfef4]  // ebp-0x8 = i (counter/file_pos)
► 0x44a87c   mov    byte ptr [ecx + 0x5415b8], al        [0x5415b8] <= 0xde  // buffer[i] = file[i]; 0x5415b8 = buffer
  0x44a882   call   0x401000        <0x401000>

  0x44a887   mov    edx, dword ptr [ebp - 8]        EDX, [0xfcfef4]
  0x44a88a   add    edx, 1        EDX => 1 (0 + 1)
► 0x44a88d   mov    dword ptr [ebp - 8], edx        [0xfcfef4] <= 1  // i += 1
  0x44a890   call   0x401000        <0x401000>

  0x44a895   jmp    0x44a856        <0x44a856>
    ↓
  0x44a856   mov    edx, dword ptr [ebp - 4]        EDX, [0xfcfef8]
  0x44a859   push   edx
► 0x44a85a   call   dword ptr [0x44c0c0]        // FUNÇÃO: feof

► 0x44a860   add    esp, 4        ESP => 0xfcfecc (0xfcfec8 + 0x4)
  0x44a863   test   eax, eax        0 & 0     EFLAGS => 0x246 [ cf PF af ZF sf IF df of ac ]
  0x44a865 ✘ jne    0x44a897        <0x44a897>  // Loop de novo; while (!feof(file))

... (loop) // --> Lê todo o arquivo para o buffer em 0x5415b8

► 0x44a897   call   0x401000        <0x401000>
```

Aqui o programa leu todo o arquivo para um buffer localizado em `0x5415b8`. Continuemos... nada de nosso input fazer efeito ainda.

```
-------------------------[ DISASM / i386 / set emulate on ]-------------------------
► 0x44a89c   mov    dword ptr [ebp - 8], 0        [0xfcfef4] <= 0  // i = 0
  0x44a8a3   jmp    0x44a8ae        <0x44a8ae>
    ↓
  0x44a8ae   mov    ecx, dword ptr [ebp - 8]        ECX, [0xfcfef4]
► 0x44a8b1   cmp    ecx, dword ptr [ebp - 0x10]        EFLAGS => 0x287 [ CF PF af zf SF IF df of ac ]
  0x44a8b4 ✘ jae    0x44a8ff        <0x44a8ff>  // if (i >= file_size)

  0x44a8b6   mov    edx, dword ptr [ebp - 8]        EDX, [0xfcfef4]  // edx <- i
// movsx = mov sign-extend
► 0x44a8b9   movsx  ecx, byte ptr [edx + 0x5415b8]        ECX, [0x5415b8]  // ecx <- buffer[i]
  0x44a8c0   mov    eax, dword ptr [ebp - 8]        EAX, [0xfcfef4]  // eax <- i
  0x44a8c3   xor    edx, edx        EDX => 0
// div: edx|eax (concatenados) -> eax * divisor + edx
► 0x44a8c5   div    dword ptr [ebp - 0xc]        // edx <- i % l
► 0x44a8c8   movsx  edx, byte ptr [edx + 0x44d370]        EDX, [0x44d370]  // edx <- input[i%l]
► 0x44a8cf   xor    ecx, edx        ECX => 33 (0x40 ^ 0x61)  // ecx <- ecx ^ edx
  0x44a8d1   mov    eax, dword ptr [ebp - 8]        EAX, [0xfcfef4]  // eax <- i
  0x44a8d4   mov    byte ptr [eax + 0x5415b8], cl        [0x5415b8] <= 0xbf  // buffer[i] ^= input[i%l]; finalmente nosso input
  0x44a8da   call   0x401000        <0x401000>

► 0x44a8df   mov    ecx, dword ptr [ebp - 8]        ECX, [0xfcfef4]  // ecx <- i
  0x44a8e2   movsx  edx, byte ptr [ecx + 0x5415b8]        EDX, [0x5415b8]  // edx <- buffer[i]
  0x44a8e9   xor    edx, 0xff        EDX => 222 (0x21 ^ 0xff)
  0x44a8ef   mov    eax, dword ptr [ebp - 8]        EAX, [0xfcfef4]
  0x44a8f2   mov    byte ptr [eax + 0x5415b8], dl        [0x5415b8] <= 0x40  // buffer[i] ^= 0xff
  0x44a8f8   call   0x401000        <0x401000>

  0x44a8fd   jmp    0x44a8a5        <0x44a8a5>
    ↓
  0x44a8a5   mov    eax, dword ptr [ebp - 8]        EAX, [0xfcfef4]
  0x44a8a8   add    eax, 1        EAX => 1 (0 + 1)
  0x44a8ab   mov    dword ptr [ebp - 8], eax        [0xfcfef4] <= 1  // i += 1
  0x44a8ae   mov    ecx, dword ptr [ebp - 8]        ECX, [0xfcfef4]
  0x44a8b1   cmp    ecx, dword ptr [ebp - 0x10]        EFLAGS => 0x283 [ CF pf af zf SF IF df of ac ]
► 0x44a8b4 ✘ jae    0x44a8ff        <0x44a8ff>  // while (i < file_size)

... (loop) // --> buffer[i] ^= input[i%l] ^ 0xff, por todos os 0x2400 bytes do arquivo

  0x44a8ff   mov    ecx, dword ptr [ebp - 4]        ECX, [0xfcfef8]
  0x44a902   push   ecx        // FILE*
► 0x44a903   call   dword ptr [0x44c0a0]        // FUNÇÃO: fclose
```

```
  0x44a909   add    esp, 4        ESP => 0xfcfecc (0xfcfec8 + 0x4)  // Limpa arg
  0x44a90c   call   0x401000        <0x401000>
```

Achamos, enfim, o que o programa faz com o arquivo. Para todo byte do arquivo, ele faz:

```
file[i] = file[i] ^ key[i%l] ^ 0xff
```

onde `l` é o tamanho de nossa chave (`key`).

Dito isso, sabemos que o *ransomware* "encripta" apenas com um XOR. Para reverter, então, basta que tenhamos alguma informação do *plaintext* para recuperar a chave utilizada. Releiamos o *readme.txt*:

```
Decrypt File (EXE)
```

Sabendo que é um XOR e que precisamos de uma dica para decriptar, eis nossa dica: *file* é do formato `EXE`! Ou seja, sabendo disso e que todo `EXE` possui um cabeçalho padrão, basta fazer um XOR do início do *file* com o cabeçalho padrão de um `EXE` e podemos extrair a chave utilizada!

Porque é a melhor linguagem existente, faremos em C. Primeiro, precisamos de um cabeçalho `EXE`, que podemos extrair do próprio *run.exe*:

```
head -c 120 run.exe > exe_header.txt
```

Então, simplesmente fazemos um código simples que desfaça o XOR:

```c
#include<stdio.h>

int main() {
  FILE* plaintxt = fopen("exe_header.txt", "rb");
  FILE* ciphertxt = fopen("file", "rb");
  int p, c, key;
  while (!feof(plaintxt)) {
    p = getc(plaintxt);
    c = getc(ciphertxt);
    key = p ^ c ^ 0xff;
    printf("%c", key);
  }
  fclose(plaintxt);
  fclose(ciphertxt);
  return 0;
}
```

Observação importantíssima: Rode a solução sobre o *file* original, pois cada execução do *run.exe* vai ter alterado o *file*.

Uma vez com o código pronto, basta rodar e ver a chave =D

```
┌──(dleskow㉿ganesh)-[/home/ganesh/Reversin.Kr/Ramsomware]
└─# ./solve
letsplaychessletsplaychessletsplaychesslet
```





E temos nosso EXE original =D

Agora, basta rodar o executável para ele te dizer a resposta:

```
C:/home> wine file
```

Se, porém, o seu Wine também der problema, como o meu, mais vale ler a resposta pelo IDA que tentar ajeita o Wine... apenas lembre de descompactar antes:

```
┌──(dleskow㉿ganesh)-[/home/ganesh/Reversin.Kr/Ransomware]
└─# upx -d file
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2024
UPX 4.2.4       Markus Oberhumer, Laszlo Molnar & John Reiser    May 9th 2024

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
     28160 <-      9216   32.73%    win32/pe     file

Unpacked 1 file.
```

E agora, no IDA:

```
int wmain_0()
{
  printf("Key -> Colle System"); /* Eis nossa resposta =D */
  getch();
  return 0;
}
```